

KODE

```
# Data Barang: Nama, Berat, dan Keuntungan
```

```
items = [  
    ("Pakaian", 2, 12),  
    ("Tas", 3, 18),  
    ("Sepatu", 4, 22),  
    ("Aksesori", 1, 8)  
]
```

```
# Fungsi untuk Dynamic Programming Knapsack
```

```
def knapsack_dp(items, capacity):
```

```
    n = len(items)
```

```
    # Tabel DP untuk menyimpan hasil sementara
```

```
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
```

```
    # Mengisi tabel DP
```

```
    for i in range(1, n + 1):
```

```
        item_name, item_weight, item_value = items[i - 1]
```

```
        for w in range(capacity + 1):
```

```
            if item_weight <= w:
```

```
                # Pilih nilai maksimum antara memilih atau tidak memilih barang
```

```
                dp[i][w] = max(dp[i - 1][w], item_value + dp[i - 1][w - item_weight])
```

```
            else:
```

```
                dp[i][w] = dp[i - 1][w]
```

```

# Menentukan barang yang dipilih
w = capacity
selected_items = []
total_weight = 0
for i in range(n, 0, -1):
    if dp[i][w] != dp[i - 1][w]:
        selected_items.append(items[i - 1][0]) # Menyimpan nama barang
        total_weight += items[i - 1][1] # Menambahkan berat barang yang dipilih
        w -= items[i - 1][1] # Mengurangi kapasitas sisa

# Nilai maksimum yang dapat diperoleh
max_value = dp[n][capacity]

return max_value, selected_items, total_weight

```

Kapasitas Knapsack

```
capacity = 20
```

Menjalankan fungsi knapsack

```
max_value, selected_items, total_weight = knapsack_dp(items, capacity)
```

Menampilkan hasil

```
print("Nilai maksimum yang dapat diperoleh:", max_value)
```

```
print("Barang yang dipilih:", selected_items)
```

```
print("Total berat barang yang dipilih:", total_weight)
```

```
n algoritma\tempCodeRunnerFile.py"
Nilai maksimum yang dapat diperoleh: 60
Barang yang dipilih: ['Aksesori', 'Sepatu',
'Tas', 'Pakaian']
Total berat barang yang dipilih: 10
PS D:\desain algoritma> & "D:/desain algori
```